

ASSIGNMENT-1

OUTPUT:

```
(venv) C:\dev\ml-project\College-ml-code\assignment-1>python ffn_mnist.py
2026-02-09 21:21:18.368793: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-02-09 21:21:21.424790: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 6s 1us/step
C:\dev\ml-project\venv\Lib\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-02-09 21:21:33.089532: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

```
Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
1688/1688 2s 738us/step - accuracy: 0.9261 - loss: 0.2541 - val_accuracy: 0.9698 - val_loss: 0.1039
Epoch 2/10
1688/1688 1s 641us/step - accuracy: 0.9685 - loss: 0.1070 - val_accuracy: 0.9728 - val_loss: 0.0935
Epoch 3/10
1688/1688 1s 637us/step - accuracy: 0.9774 - loss: 0.0720 - val_accuracy: 0.9770 - val_loss: 0.0833
Epoch 4/10
1688/1688 1s 632us/step - accuracy: 0.9822 - loss: 0.0547 - val_accuracy: 0.9807 - val_loss: 0.0692
Epoch 5/10
1688/1688 1s 637us/step - accuracy: 0.9855 - loss: 0.0441 - val_accuracy: 0.9762 - val_loss: 0.0846
Epoch 6/10
1688/1688 1s 635us/step - accuracy: 0.9894 - loss: 0.0352 - val_accuracy: 0.9793 - val_loss: 0.0755
Epoch 7/10
1688/1688 1s 641us/step - accuracy: 0.9901 - loss: 0.0291 - val_accuracy: 0.9783 - val_loss: 0.0825
Epoch 8/10
1688/1688 1s 639us/step - accuracy: 0.9918 - loss: 0.0242 - val_accuracy: 0.9785 - val_loss: 0.0828
Epoch 9/10
1688/1688 1s 650us/step - accuracy: 0.9932 - loss: 0.0207 - val_accuracy: 0.9785 - val_loss: 0.0921
Epoch 10/10
1688/1688 1s 651us/step - accuracy: 0.9929 - loss: 0.0201 - val_accuracy: 0.9790 - val_loss: 0.0903
Test Accuracy: 0.978608025177002
```

CONCLUSION: This experiment compared Batch Gradient Descent, Stochastic Gradient Descent, and Mini Batch Gradient Descent on the same neural network model. The difference in training behavior was clear. Batch GD was stable but slow, SGD was noisy but faster, and Mini Batch gave the best balance between speed and convergence. From this, Mini Batch Gradient Descent proved to be the most practical method for real world training.

ASSIGNMENT-2

OUTPUT:

```
(venv) C:\dev\ml-project\College-ml-code\assignment-2>python gradient_descent_comparison.py
2026-02-09 21:38:19.375897: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-02-09 21:38:20.363557: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.

===== Batch Gradient Descent =====
C:\dev\ml-project\venv\Lib\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-02-09 21:38:21.074146: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Batch GD Accuracy: 0.9086115956386458

===== Stochastic Gradient Descent =====
SGD Accuracy: 1.0

===== Mini-Batch Gradient Descent =====
Mini-Batch GD Accuracy: 0.9736379384994507

===== Final Comparison =====
Batch GD Accuracy      : 0.9086
SGD Accuracy           : 1.0000
Mini-Batch GD Accuracy : 0.9736
```

CONCLUSION: In this assignment a feed forward neural network was implemented and trained on the MNIST handwritten digit dataset. The model was able to learn digit patterns effectively and achieved high classification accuracy. This demonstrated how multilayer perceptrons can solve image classification problems when proper preprocessing and architecture design are applied.

ASSIGNMENT-3

OUTPUT:

alexnet

```
(venv) C:\dev\ml-project\College-ml-code\assignment-3>python alexnet.py
2026-02-09 22:22:01.776723: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-02-09 22:22:03.842894: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Found 796 images belonging to 4 classes.
Found 171 images belonging to 4 classes.
Found 173 images belonging to 4 classes.
C:\dev\ml-project\venv\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-02-09 22:22:04.375966: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Epoch 1/5
25/25 ----- 13s 473ms/step - accuracy: 0.2613 - loss: 1.9389 - val_accuracy: 0.2632 - val_loss: 1.3860
Epoch 2/5
25/25 ----- 12s 468ms/step - accuracy: 0.2613 - loss: 1.3904 - val_accuracy: 0.2632 - val_loss: 1.3821
Epoch 3/5
25/25 ----- 11s 456ms/step - accuracy: 0.2513 - loss: 1.3852 - val_accuracy: 0.2632 - val_loss: 1.3821
Epoch 4/5
25/25 ----- 11s 456ms/step - accuracy: 0.2500 - loss: 1.3854 - val_accuracy: 0.2632 - val_loss: 1.3822
Epoch 5/5
25/25 ----- 11s 457ms/step - accuracy: 0.2198 - loss: 1.3841 - val_accuracy: 0.2632 - val_loss: 1.3823
6/6 ----- 1s 79ms/step - accuracy: 0.2659 - loss: 1.3828
Accuracy: 0.2658959627151489
6/6 ----- 1s 82ms/step
[[ 0 45  0  0]
 [ 0 46  0  0]
 [ 0 45  0  0]
 [ 0 37  0  0]]
C:\dev\ml-project\venv\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\dev\ml-project\venv\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\dev\ml-project\venv\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
      precision    recall  f1-score   support

         0         0.00      0.00      0.00         45
         1         0.27      1.00      0.42         46
         2         0.00      0.00      0.00         45
         3         0.00      0.00      0.00         37

   accuracy          0.27         0.27      173
  macro avg          0.07         0.25         0.11      173
 weighted avg          0.07         0.27         0.11      173
```

mobilenet

```
(venv) C:\dev\ml-project\College-ml-code\assignment-3>python mobilenet.py
2026-02-09 22:00:14.949918: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-02-09 22:00:16.120643: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Found 796 images belonging to 4 classes.
Found 171 images belonging to 4 classes.
Found 173 images belonging to 4 classes.
2026-02-09 22:00:17.536285: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVXS12F AVXS12F VNNI AVXS12F BF16 AVX VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 — 14s 2us/step
Epoch 1/5
25/25 — 14s 527ms/step - accuracy: 0.8103 - loss: 4.6845 - val_accuracy: 0.9474 - val_loss: 0.5700
Epoch 2/5
25/25 — 10s 397ms/step - accuracy: 0.9686 - loss: 0.5816 - val_accuracy: 0.9591 - val_loss: 0.5029
Epoch 3/5
25/25 — 10s 385ms/step - accuracy: 0.9786 - loss: 0.2314 - val_accuracy: 0.9649 - val_loss: 0.8131
Epoch 4/5
25/25 — 10s 381ms/step - accuracy: 0.9887 - loss: 0.2336 - val_accuracy: 0.9649 - val_loss: 0.5604
Epoch 5/5
25/25 — 10s 383ms/step - accuracy: 0.9962 - loss: 0.0285 - val_accuracy: 0.9415 - val_loss: 0.9455
6/6 — 2s 284ms/step - accuracy: 0.9769 - loss: 0.2943
Accuracy: 0.9768785834312439
6/6 — 2s 279ms/step
[[44  1  0  0]
 [ 0 45  1  0]
 [ 0  1 44  0]
 [ 0  0  1 36]]
      precision    recall  f1-score   support

     0         1.00      0.98      0.99         45
     1         0.96      0.98      0.97         46
     2         0.96      0.98      0.97         45
     3         1.00      0.97      0.99         37

 accuracy          0.98
 macro avg         0.98
 weighted avg      0.98
```

resnet152

```
(venv) C:\dev\ml-project\College-ml-code\assignment-3>python resnet152.py
2026-02-09 22:24:26.114943: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-02-09 22:24:27.235411: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Found 796 images belonging to 4 classes.
Found 171 images belonging to 4 classes.
Found 173 images belonging to 4 classes.
2026-02-09 22:24:28.476510: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVXS12F AVXS12F VNNI AVXS12F BF16 AVX VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet152_weights_tf_dim_ordering_tf_kernels_notop.h5
234608864/234608864 — 18s 0us/step
Epoch 1/5
25/25 — 45s 2s/step - accuracy: 0.2764 - loss: 5.2559 - val_accuracy: 0.2924 - val_loss: 1.4384
Epoch 2/5
25/25 — 37s 1s/step - accuracy: 0.3643 - loss: 1.4311 - val_accuracy: 0.4386 - val_loss: 1.1340
Epoch 3/5
25/25 — 37s 1s/step - accuracy: 0.4535 - loss: 1.2616 - val_accuracy: 0.2690 - val_loss: 1.5739
Epoch 4/5
25/25 — 37s 1s/step - accuracy: 0.4359 - loss: 1.2677 - val_accuracy: 0.4737 - val_loss: 1.4083
Epoch 5/5
25/25 — 37s 1s/step - accuracy: 0.5528 - loss: 1.0387 - val_accuracy: 0.6199 - val_loss: 0.9754
6/6 — 6s 931ms/step - accuracy: 0.5549 - loss: 0.9931
Accuracy: 0.5549132823944092
6/6 — 9s 1s/step
[[41  0  4  0]
 [14 20 12  0]
 [18  2 25  0]
 [11  0 16 10]]
      precision    recall  f1-score   support

     0         0.49      0.91      0.64         45
     1         0.91      0.43      0.59         46
     2         0.44      0.56      0.49         45
     3         1.00      0.27      0.43         37

 accuracy          0.55
 macro avg         0.71
 weighted avg      0.70
```

Vgg16

```
(venv) C:\dev\ml-project\college-ml-code\assignment-3>python vgg16.py
2026-02-09 22:12:29.470884: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-02-09 22:12:30.609852: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Found 796 images belonging to 4 classes.
Found 171 images belonging to 4 classes.
Found 173 images belonging to 4 classes.
C:\dev\ml-project\venv\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-02-09 22:12:31.863591: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Epoch 1/5
25/25 ━━━━━━━━━━━ 39s 2s/step - accuracy: 0.2952 - loss: 3.1244 - val_accuracy: 0.4211 - val_loss: 1.3602
Epoch 2/5
25/25 ━━━━━━━━━━━ 38s 2s/step - accuracy: 0.3379 - loss: 1.3222 - val_accuracy: 0.4035 - val_loss: 1.1935
Epoch 3/5
25/25 ━━━━━━━━━━━ 37s 1s/step - accuracy: 0.4560 - loss: 1.1824 - val_accuracy: 0.4620 - val_loss: 1.1696
Epoch 4/5
25/25 ━━━━━━━━━━━ 39s 2s/step - accuracy: 0.4925 - loss: 1.0722 - val_accuracy: 0.4035 - val_loss: 1.0637
Epoch 5/5
25/25 ━━━━━━━━━━━ 39s 2s/step - accuracy: 0.4862 - loss: 1.0965 - val_accuracy: 0.5439 - val_loss: 1.1054
6/6 ━━━━━━━━━━━ 2s 243ms/step - accuracy: 0.5260 - loss: 1.0098
Accuracy: 0.52601158618927
6/6 ━━━━━━━━━━━ 2s 246ms/step
[[ 45  0  0  0]
 [ 29 17  0  0]
 [ 15  1  0 29]
 [  8  0  0 29]]
C:\dev\ml-project\venv\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\dev\ml-project\venv\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\dev\ml-project\venv\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
      precision    recall  f1-score   support

     0       0.46       1.00       0.63        45
     1       0.94       0.37       0.53        46
     2       0.00       0.00       0.00        45
     3       0.50       0.78       0.61        37

 accuracy          0.53        173
 macro avg         0.48         0.54         0.44        173
 weighted avg         0.48         0.53         0.44        173
```

CONCLUSION: This work focused on classifying microplastic images using multiple deep learning architectures. The dataset was preprocessed and split into training, validation, and testing sets before model training. Different architectures showed variation in accuracy and performance metrics, highlighting the impact of model depth and transfer learning. The evaluation using confusion matrix, precision, recall, and F1 score provided a complete view of model effectiveness.